Section 13 - OOP - Classes and Objects

Information

- Title: `Section 13 - OOP - Classes and Objects`
- Authors: `Etienne KOA`

Aperçu

- Qu'est-ce que la programmation orientée objet ?
- Que sont les classes et les objets ?
- Déclarer le classement et créer des objets
- Opérateurs de points et de pointeurs
- Modificateurs d'accès « public » et « privé »
- Méthodes, Constructeurs et Destructeurs
  - méthodes `class`
  - constructeurs surchargés par défaut
  - copier et déplacer les constructeurs
  - copie superficielle vs copie profonde
  - pointeur `this`
- Membres de la classe « statiques »
- `struct` contre `class`
- `friend` d'une classe

Qu'est-ce que la programmation orientée objet ?

Qu'est-ce que la programmation orientée objet ?

- Programmation procédurale
- Limites de la programmation procédurale
- OO Concepts de programmation et leurs avantages
- OO Limites de programmation

Programmation procédurale

- L'accent est mis sur les processus ou les actions qu'un programme entreprend
- Les programmes sont généralement un ensemble de fonctions
- Les données sont déclarées séparément
- Les données sont transmises sous forme d'arguments aux fonctions
- Assez facile à apprendre

Limites de la programmation procédurale

- Les fonctions doivent connaître la structure des données
  - Si la structure des données change, de nombreuses fonctions doivent être modifiées
- À mesure que les programmes grandissent, ils deviennent plus nombreux :

  - difficile à comprendre
  - difficile à entretenir
  - difficile à prolonger
  - difficile à déboguer
  - code difficile à réutiliser
  - fragile et plus facile à casser

Qu'est-ce que la programmation orientée objet ?

- Classes et objets
  - L'accent est mis sur les classes qui modélisent des entités de domaine du monde réel
  - permet aux développeurs de penser à un niveau d'abstraction plus élevé
  - utilisé avec succès dans de très gros programmes
- Encapsulation
  - Les objets contiennent des données ET des opérations qui fonctionnent sur ces données
  - Type de données abstrait (ADT)
- Masquage d'informations
  - La logique spécifique à l'implémentation peut être masquée
  - Utilisateurs du code de classe à l'interface puisqu'ils n'ont pas besoin de connaître l'implémentation
  - plus d'abstraction
  - plus facile à tester, déboguer, maintenir et étendre
- Réutilisabilité
  - plus facile de réutiliser les classes dans d'autres applications
  - développement plus rapide
  - qualité supérieure
- Héritage
  - peut créer de nouvelles classes à partir de classes existantes
  - réutilisabilité
  - classes polymorphes
- Polymorphosme et plus encore...

Limites

- Pas une panacée
  - La programmation OO n'améliorera pas le mauvais code
  - ne convient pas à tous types de problèmes
  - tout ne se décompose pas en une classe
- Courbe d'apprentissage

  - généralement une courbe d'apprentissage plus raide, en particulier pour `C++`
  - de nombreux langages OO, de nombreuses variantes de concepts OO
- Conception
  - Habituellement, une conception plus initiale est nécessaire pour créer de bons modèles et hiérarchies
- Les programmes peuvent être :
  - taille plus grande
  - plus lent
  - plus complexe

Que sont les classes et les objets ?

Classes et objets

- **Cours**
  - plan à partir duquel les objets sont créés
  - un type de données défini par l'utilisateur
  - a des attributs (données)
  - a des méthodes (fonctions)
  - peut masquer les données et les méthodes
  - fournit une interface publique
- **Exemples de cours**
  - Account
  - Employee
  - Image
  - `std::vector`
  - `std::string`
- **Objets**
  - créé à partir d'une classe
  - représente une instance spécifique d'une classe
  - peut traiter de très nombreux objets
  - chacun a sa propre identité
  - chacun peut utiliser les méthodes de classe définies

Classes et objets

```
int high_score;
int low_score;

Account frank_account;
Account jim_account;

std::vector<int> scores;
std::string name;
```

```
int high_score;
int low_score;

Account frank_account;
Account jim_account;

std::vector<int> scores;
std::string name;
```

Figure 1: OOP

Déclarer une classe et créer des objets

Déclarer une classe

```
class Class_Name
{

  // declaration(s);

};
```

Figure 2: OOP

```
class Class_Name
{
  // declaration (s);
}
```

Player

```
class Player
{
  // attributes
  std:string name;
  int health;
  int xp;

  // methods
  void talk (std::string, text_to_say){
    bool_is_dead();
  }
};
```

Création d'objets

```
class Player
{
   // attributes
   std::string name;
   int health;
   int xp;

  // methods
  void talk(std::string text_to_say);
   bool is_dead();
};
```

Figure 3: OOP

```
Player frank;
Player hero;


Player *enemy = new Player();
delete enemy;
```

Figure 4: OOP

```
Player frank;
Player hero;

Player *enemy = new Player();
delete enemy;
```

Account

```
class Account
{
  // attributes
  std:string name;
  double balance;

  // methods
  bool withdraw(double amount);
  bool deposit(double amount);
};
```

Création d'objets

```
class Account
{
   // attributes
   std::string name;
   double balance;

   // methods
   bool withdraw(double amount);
   bool deposit(double amount);
};
```

Figure 5: OOP

```
Account frank_account;
Account jim_account;


Account *mary_account = new Account();
delete mary_account;

Account frank_account;
Account jim_account;

Account accounts[] {frank_account, jim_account};

std::vector<Account> accounts1 {frank_account};
accounts1.push_back(jim_account);
```

```
Account frank_account;
Account jim_account;

Account *mary_account = new Account();
delete mary_account;

Account frank_account;
Account jim_account;

Accounts accounts[] {frank_account, jim_account};

std::vector<Account> accounts1 {frank_account};
accounts1.push_back(jim_account);
```

Accessing Class Members

- Nous pouvons accéder
  - attributs de classe
  - méthodes de classe
- Certains membres de la classe ne seront pas accessibles (nous en reparlerons plus tard)
- Nous avons besoin d'un objet pour accéder aux variables d'instance

Si nous avons un objet (opérateur point)

- **Utilisation de l'opérateur point**

```
Account frank_account;

frank_account.balance;
frank_account.deposit(1000.00);
```

Figure 6: OOP

```
Account frank_account;

frank_account.balance;
frank_account.deposit(1000.00);
```

Si nous avons un pointeur vers un objet (membre de l'opérateur pointeur)

- Déréférencez le pointeur puis utilisez l'opérateur point.

```
Account  *frank_account = new Account();

(*frank_account).balance;
(*frank_account).deposit(1000.00);
```

Figure 7: OOP

```
Account *frank_account = new Account();

(*frank_account).balance;
(*frank_account).deposit(1000.00);
```

- Ou utilisez l'opérateur membre de pointeur (opérateur flèche)

```
Account * frank_account = new Account();

frank_account -> balance;
frank_account -> deposit(1000.00);
```

```
Account *frank_account = new Account();

frank_account->balance;
frank_account->deposit(1000.00);
```

Figure 8: OOP

Publique et privée

- **public, privé et protégé**
- `public`
  - accessible partout

```
class Class_Name
{
public:

  // declaration(s);

};
```

Figure 9: OOP

```
class Class_Name
{
  public :

  // declaration(s);
};
```

- `private`
  - accessible uniquement aux membres ou amis de la classe

```
class Class_Name
{
  private :

  // declaration(s);
};
```

- `protected`
  - utilisé avec l'héritage - nous en parlerons dans la section suivante

```
class Class_Name
```

```
class Class_Name
{
private:

  // declaration(s);

};
```

Figure 10: OOP

```
class Class_Name
{
protected:

  // declaration(s);

};
```

Figure 11: OOP

```
{
  protected :

  // declaration(s);
};
```

Déclarer une classe

- **Player**

Création d'objets

Déclarer une classe

- **Account**

Création d'objets

Implémentation des méthodes membres

- Très similaire à la façon dont nous avons implémenté les fonctions
- Les méthodes membres ont accès aux attributs des membres
  - Vous n'avez donc pas besoin de les passer en arguments !

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  void talk(std::string text_to_say);
   bool is_dead();
};
```

Figure 12: OOP

```
Player frank;
frank.name = "Frank";          // Compiler error
frank.health = 1000;           // Compiler error
frank.talk("Ready to battle");     // OK


Player *enemy = new Player();
enemy->xp = 100;               // Compiler error
enemy->talk("I will hunt you down"); // OK

delete enemy;
```

Figure 13: OOP

```
class Account
{
private:
   std::string name;
   double balance;

public:
  bool withdraw(double amount);
  bool deposit(double amount);
};
```

Figure 14: OOP

```
Account frank_account;
frank_account.balance = 10000000.00; // Compiler error
frank_account.deposit(10000000.0);        // OK
frank_account.name = "Frank's Account";   // Compiler error

Account *mary_account = new Account();

mary_account->balance = 10000.0;  // Compiler error
mary_account->withdraw(10000.0);  // OK

delete mary_account;
```

Figure 15: OOP

- Peut être implémenté dans la déclaration de classe
  - Implicite en ligne
- Peut être implémenté en dehors de la déclaration de classe
  - Besoin d'utiliser `Class_name ::method_name`
- Peut séparer la spécification de la mise en œuvre
  - Fichier `.h` pour la déclaration de classe
  - Fichier `.cpp` pour l'implémentation de la classe

À l'intérieur de la déclaration de classe

```
class Account {

private:
   double balance;
public:
   void set_balance(double bal) {
      balance = bal;
   }
   double get_balance() {
      return balance;
   }
};
```

Figure 16: OOP

En dehors de la déclaration de classe

Séparer la spécification de la mise en œuvre

- Inclure les gardes

```
class Account {

private:
   double balance;
public:
   void set_balance(double bal);
   double get_balance();
};

void Account::set_balance(double bal) {
   balance = bal;
}
double Account::get_balance() {
   return balance;
}
```

Figure 17: OOP

Account.h

```
class Account {

private:
   double balance;
public:
   void set_balance(double bal);
   double get_balance();
};
```

Figure 18: OOP

```
#ifndef _ACCOUNT_H_
#define _ACCOUNT_H_

   // Account class declaration

#endif
```

Account.h

```
#ifndef _ACCOUNT_H_
#define _ACCOUNT_H_

class Account {

private:
   double balance;
public:
   void set_balance(double bal);
   double get_balance();
};

#endif
```

Account.h - #pragma once

```
#pragma once

class Account {

private:
   double balance;
public:
   void set_balance(double bal);
   double get_balance();
};
```

Account.cpp

```
#include "Account.h"

void Account::set_balance(double bal) {
   balance = bal;
}

double Account::get_balance() {
   return balance;
}
```

```
main.cpp
#include <iostream>
#include "Account.h"

int main() {
   Account frank_account;
   frank_account.set_balance(1000.00);
   double bal = frank_account.get_balance();

   std::cout << bal << std::endl; // 1000
   return 0;
}
```

Constructeurs

- Méthode membre spécial
- Invoqué lors de la création de l'objet
- Utile pour l'initialisation
- Même nom que la classe
- Aucun type de retour n'est spécifié
- Peut être surchargé

Constructeurs de joueurs

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  // Overloaded Constructors
  Player();
  Player(std::string name);
  Player(std::string name, int health, int xp);
};
```

Figure 19: OOP

Constructeurs de comptes

Destructeurs

- Méthode membre spécial
- Même nom car la classe a procédé avec un tilde (~)
- Invoqué automatiquement lorsqu'un objet est détruit
- Aucun type de retour et aucun paramètre

```
class Account
{
private:
   std::string name;
   double balance;
public:
  // Constructors
  Account();
  Account(std::string name, double balance);
  Account(std::string name);
  Account(double balance);
};
```

Figure 20: OOP

- Un seul destructeur est autorisé par classe - ne peut pas être surchargé !
- Utile pour libérer de la mémoire et d'autres ressources

Player Destructor

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  Player();
  Player(std::string name);
  Player(std::string name, int health, int xp);
  // Destructor
  ~Player();
};
```

Account Destructor

```
class Account
{
private:
   std::string name;
   double balance;
public:
  Account();
  Account(std::string name, double balance);
  Account(std::string name);
  Account(double balance);
  // Destructor
  ~Account();
};
```

```
{
   Player slayer;
   Player frank {"Frank", 100, 4 };
   Player hero {"Hero"};
   Player villain {"Villain"};
   // use the objects
}  // 4 destructors called

Player *enemy = new Player("Enemy", 1000, 0);
delete enemy; // destructor called
```

Creating objects

Le constructeur par défaut

- N'attend aucun argument
  - Également appelé constructeur sans arguments
- Si vous n'écrivez aucun constructeur pour une classe
  - `C++` générera un `Default Constructor` qui ne fait rien
- Appelé lorsque vous instanciez un nouvel objet sans argument

```
Player frank;
Player *enemy = new Player;
```

Figure 21: OOP

```
Player frank;
Player *enemy = new Player;
```

Déclarer une classe + Compte - en utilisant le constructeur par défaut

```
class Account
{
private:
   std::string name;
   double balance;
public:
  bool withdraw(double amount);
  bool deposit(double amount);
};
```

Figure 22: OOP

Création d'objets + Utilisation du constructeur par défaut

Déclarer une classe + Compte - constructeur sans argument défini par l'utilisateur

```
Account frank_account;
Account jim_account;


Account *mary_account = new Account;
delete mary_account;
```

Figure 23: OOP

```
class Account
{
private:
   std::string name;
   double balance;
public:
   Account() {
      name = "None";
      balance = 0.0;
   }
  bool withdraw(double amount);
  bool deposit(double amount);
};
```

Figure 24: OOP

- Compte - pas de constructeur par défaut

Création d'objets + Utilisation du constructeur par défaut

Surcharge des constructeurs

- Les classes peuvent avoir autant de constructeurs que nécessaire.
- Chacun doit avoir une signature unique
- Le constructeur par défaut n'est plus généré par le compilateur une fois qu'un autre constructeur est déclaré.

Constructeurs surchargés

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  // Overloaded Constructors
  Player();
  Player(std::string name_val);
  Player(std::string name_val, int health_val, int xp_val);
};
```

```
class Account
{
private:
   std::string name;
   double balance;
public:
   Account(std::string name_val, double bal) {
      name = name_val;
      balance = bal;
   }
  bool withdraw(double amount);
  bool deposit(double amount);
};
```

Figure 25: OOP

```
Account frank_account;                    // Error
Account jim_account;                      // Error

Account *mary_account = new Account;    // Error
delete mary_account;

Account bill_account {"Bill", 15000.0}; // OK
```

Figure 26: OOP

```
Player::Player() {
   name = "None";
   health = 0;
   xp = 0;
}

Player::Player(std::string name_val) {
   name = name_val;
   health = 0;
   xp = 0;
}

Player::Player(std::string name_val, int health_val, int
xp_val) {
   name = name_val;
   health = health_val;
   xp = xp_val;
}
```

Création d'objets

```
Player empty;                                  // None, 0, 0

Player hero {"Hero"};                         // Hero, 0, 0
Player villain {"Villain"};                  // Villain, 0, 0

Player frank {"Frank", 100, 4};             // Frank, 100, 4

Player *enemy = new Player("Enemy", 1000, 0); // Enemy, 1000, 0
delete enemy;
```

Listes d'initialisation du constructeur

- Jusqu'à présent, toutes les valeurs des membres de données ont été définies dans le corps du constructeur
- Listes d'initialisation du constructeur
  - sont plus efficaces
  - la liste d'initialisation suit immédiatement la liste des paramètres
  - initialise les données membres lors de la création de l'objet !
  - l'ordre d'initialisation est l'ordre de déclaration dans la classe

Player()

Player(std::string)

Player(std::string, int, int)

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  // Overloaded Constructors
  Player();
  Player(std::string name_val);
  Player(std::string name_val, int health_val, int xp_val);
};
```

Figure 27: OOP

```
Previous way:
Player::Player() {
   name = "None";    // assignment not initialization
   health = 0;
   xp = 0;
}

Better way:
Player::Player()
   : name{"None"}, health{0}, xp{0} {
}
```

Figure 28: OOP

```
Previous way:
Player::Player(std::string name_val) {
   name = name_val;    // assignment not initialization
   health = 0;
   xp = 0;
}

Better way:
Player::Player(std::string name_val)
   : name{name_val}, health{0}, xp{0} {
}
```

Figure 29: OOP

```
Previous way:
Player::Player(std::string name_val, int health_val, int xp_val) {
   name = name_val;             // assignment not initialization
   health = health_val;
   xp = xp_val;
}

Better way:
Player::Player(std::string name_val, int health_val, int xp_val)
      : name{name_val}, health{health_val}, xp{xp_val} {
}

Player::Player()
   : name{"None"}, health{0}, xp{0} {
}

Player::Player(std::string name_val)
   : name{name_val}, health{0}, xp{0} {
}

Player::Player(std::string name_val, int health_val, int
xp_val)
      : name{name_val}, health{health_val}, xp{xp_val} {
}
```

Délégation des constructeurs

- Souvent, le code des constructeurs est très similaire
- Un code dupliqué peut entraîner des erreurs
- `C++` permet de déléguer des constructeurs
  - le code pour un constructeur peut en appeler un autre dans la liste d'initialisation
  - évite la duplication de code

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  // Overloaded Constructors
  Player();
  Player(std::string name_val);
  Player(std::string name_val, int health_val, int xp_val);
};
```

```
Player::Player()
   : name{"None"}, health{0}, xp{0} {
}

Player::Player(std::string name_val)
   : name{name_val}, health{0}, xp{0} {
}

Player::Player(std::string name_val, int health_val, int
xp_val)
   : name{name_val}, health{health_val}, xp{xp_val} {
}

Player::Player(std::string name_val, int health_val, int xp_val)
   : name{name_val}, health{health_val}, xp{xp_val} {
}

Player::Player()
   : Player {"None", 0, 0} {
}

Player::Player(std::string name_val)
   : Player { name_val, 0, 0} {
}
```

Paramètres du constructeur par défaut

- Peut souvent simplifier notre code et réduire le nombre de constructeurs surchargés.
- Les mêmes règles s'appliquent que celles que nous avons apprises avec les fonctions non membres.

```
class Player
{
private:
   std::string name;
   int health;
   int xp;
public:
  // Constructor with default parameter values
  Player(std::string name_val = "None",
         int health_val = 0,
         int xp_val = 0);
};
```

```
Player::Player(std::string name_val, int health_val, int
xp_val)
    : name {name_val}, health {health_val}, xp {xp_val} {

}
Player empty;                        // None, 0, 0
Player frank {"Frank"};                // Frank, 0, 0
Player villain {"Villain", 100, 55};  // Villain, 100, 55
Player hero {"Hero", 100};           // Hero, 100, 0

// Note what happens if you declare a no-args constructor
```

Copier le constructeur

- Lorsque les objets sont copiés, `C++` doit créer un nouvel objet à partir d'un objet existant.
- Quand fait-on une copie d'un objet ?
  - passer l'objet par valeur en paramètre
  - renvoyer un objet d'une fonction par valeur
  - construire un objet basé sur un autre de la même classe
- `C++` doit avoir un moyen d'y parvenir afin qu'il fournisse un constructeur de copie défini par le compilateur si vous ne le faites pas.

Transmettre l'objet par valeur

```
Player hero {"Hero", 100, 20};

void display_player(Player p) {
   // p is a COPY of hero in this example
   // use p
   // Destructor for p will be called
}

display_player(hero);
```

Figure 30: OOP

Renvoyer l'objet par valeur

```
Player enemy;

Player create_super_enemy() {
   Player an_enemy{"Super Enemy", 1000, 1000};
   return an_enemy;  // A COPY of an_enemy is returned
}

enemy = create_super_enemy();

Player hero {"Hero", 100, 100};

Player another_hero {hero};  // A COPY of hero is made
```

- Si vous ne fournissez pas votre propre façon de copier des objets par valeur, le compilateur fournit une méthode par défaut pour copier des objets.
- Copie les valeurs de chaque membre de données dans le nouvel objet
  - copie par défaut des membres
- Parfaitement bien dans de nombreux cas
- Attention si vous avez un membre de données de pointeur
  - Le pointeur sera copié
  - Pas ce qu'il indique
  - '`Shallow` vs. `Deep copy` - plus dans la prochaine vidéo

Les meilleures pratiques

- Fournissez un constructeur de copie lorsque votre classe a des membres de pointeur bruts
- Fournissez au constructeur de copie un paramètre `const reference`.
- Utilisez les classes `STL` car elles fournissent déjà des constructeurs de copie.
- Évitez d'utiliser des membres de données de pointeur bruts si possible.

Déclaration du constructeur de copie

```
Type::Type(const Type &source);


Player::Player(const Player &source);

Account::Account(const Account &source);
```

Figure 31: OOP

Implémentation du constructeur de copie

```
Type::Type(const Type &source) {
   // code or initialization list to copy the object
}
```

```
Player
```

```
Player::Player(const Player &source)
    : name{source.name},
      health {source.health},
      xp {source.xp} {
}
```

```
Account
```

```
Account::Account(const Account &source)
    : name{source.name},
      balance {source.balance} {
}
```

Copie Sallow avec le constructeur de copie

Copie superficielle ou profonde

- Considérez une classe qui contient un pointeur en tant que donnée membre.
- Le constructeur alloue le stockage de manière dynamique et initialise le pointeur.
- Destructor libère la mémoire allouée par le constructeur.
- Que se passe-t-il dans le constructeur de copie par défaut ?

Constructeur de copie par défaut + copie réservée aux membres + Chaque membre de données est copié à partir de l'objet source + Le pointeur n'est pas copié ce vers quoi il pointe (copie superficielle) + Problème : lors de la libération du stockage dans le constructeur, l'autre objet fait toujours référence au stockage libéré !

Peu profond

```
class Shallow {
private:
    int *data;                                  // POINTER
public:
   Shallow(int d);                             // Constructor
   Shallow(const Shallow &source);              // Copy
Constructor
   ~Shallow();                                 // Destructor
};
```

Figure 32: OOP

Constructeur peu profond

```
Shallow::Shallow(int d) {
    data = new int;     // allocate storage
    *data = d;
}
```

Destructeur peu profond

```
Shallow::~Shallow() {
    delete data;    // free storage
    cout << "Destructor freeing data" << endl;
}
```

Constructeur de copie superficielle

```
Shallow::Shallow(const Shallow &source)
    : data(source.data) {
        cout << "Copy constructor - shallow"
             << endl;
}
```

- Copie superficielle - seul le pointeur est copié - pas vers quoi il pointe !
- Problème : `source` et l'objet nouvellement créé pointent TOUS LES DEUX vers la MÊME zone de `données`.

Shallow - une méthode simple qui attend une copie

- Copie superficielle - seul le pointeur est copié - pas vers quoi il pointe !
- Problème : `source` et l'objet nouvellement créé pointent TOUS LES DEUX vers la MÊME zone de `données`.

Exemple principal - va probablement planter

Copie approfondie avec le constructeur de copie

Constructeur de copie utilisateur-fournisseur

```
Shallow::Shallow(const Shallow &source)
    : data(source.data) {
        cout << "Copy constructor - shallow"
             << endl;
}
```

Figure 33: OOP

```
int main() {
    Shallow obj1 {100};
    display_shallow(obj1);
   // obj1's data has been released!

    obj1.set_data_value(1000);
    Shallow obj2 {obj1};
    cout << "Hello world" << endl;
    return 0;
}
```

Figure 34: OOP

- Créer une « copie » des données pointées
- Chaque copie aura un pointeur vers un stockage unique dans le tas
- Copie complète lorsque vous avez un pointeur brut en tant que membre de données de classe

Profond

```
class Deep {
private:
    int *data;                // POINTER
public:
   Deep(int d);               // Constructor
   Deep(const Deep &source);     // Copy Constructor
   ~Deep();               // Destructor
};
```

Figure 35: OOP

Constructeur profond

```
Deep::Deep(int d) {
    data = new int;     // allocate storage
    *data = d;
}

Deep::~Deep() {
    delete data;    // free storage
    cout << "Destructor freeing data" << endl;
}

Deep::Deep(const Deep &source)
{
    data = new int;    // allocate storage
    *data = *source.data;
    cout << "Copy constructor - deep"
         << endl;
}
```

- Copie approfondie - créez de nouvelles valeurs de stockage et de copie

Constructeur de copie profonde - constructeur délégué

```
Deep::Deep(const Deep &source)
   : Deep{*source.data} {
        cout << "Copy constructor - deep"
             << endl;
}
```

Figure 36: OOP

- Déléguez à `Deep (int)` et transmettez la source `int (*source.data)` vers laquelle pointe.

Deep - une méthode simple qui attend une copie

```
void display_deep(Deep s) {
    cout << s.get_data_value() << endl;
}
```

Figure 37: OOP

- Lorsque `s` sort du champ d'application, le destructeur est appelé et libère `data`.
- Pas de problème : puisque le stockage en cours de publication est unique à `s`.

Exemple principal - ne plantera pas

```
int main() {
    Deep obj1 {100};
    display_deep(obj1);

    obj1.set_data_value(1000);
    Deep obj2 {obj1};

    return 0;
}
```

Figure 38: OOP

Déplacer le constructeur

- Parfois, lorsque nous exécutons du code, le compilateur crée des valeurs temporaires sans nom

```
int total {0};
total = 100 + 200;
```

- `100 + 200` est évalué `et 300` est stocké dans une valeur temporaire sans nom.
- Ensuite, `300` est ensuite stocké dans la variable total
- Ensuite, la valeur de température est supprimée
- La même chose se produit également avec les objets

Quand est-ce utile ?

- Parfois, les constructeurs de copie sont appelés plusieurs fois automatiquement en raison de la sémantique de copie vers `C++`.
- Les constructeurs de copie effectuant une copie approfondie peuvent avoir un goulot d'étranglement important en termes de performances.
- `C++11` a introduit la sémantique des déplacements et le constructeur de déplacements
- Le constructeur de déplacement déplace un objet plutôt que de le copier.
- Facultatif mais recommandé lorsque vous disposez d'un pointeur brut.
- L'élision de copie `-C++` peut optimiser complètement la copie `(RVO - Return Value Optimization)`.

références de valeur r + Utilisé dans la sémantique mobile et la transmission parfaite + La sémantique de déplacement concerne uniquement les références à la valeur R + Utilisé par le constructeur de déplacement et l'opérateur d'affectation de déplacement pour déplacer efficacement un objet plutôt que de le copier. + Opérateur de référence de valeur R (&&).

```
int x {100}
  int &l_ref = x;   // l-value reference
  l_ref = 10;       // change x to 10

  int &&r_ref = 200;    // r-value ref
  r_ref = 300;      // change r_ref to 300

  int &&x_ref = x;    // Compiler error
```

Figure 39: OOP

Paramètres de référence de valeur L

```
int x {100};         // x is an l-value

void func(int &num); // A

func(x);          // calls A - x is an l-value
func(200);           // Error - 200 is an r-value
```

Figure 40: OOP

- erreur : impossible de lier une référence lvalue non const de type `int&` à une rvalue de type `int`.

paramètres de référence de valeur r

```
int x {100};         // x is an l-value

void func(int &&num);   // B

func(200);        // calls B - 200 is an r-value
func(x);          // ERROR - x is an l-value
```

Figure 41: OOP

- erreur : impossible de lier la référence rvalue de type `int&&` à la lvalue de type `int`.

Paramètres de référence de valeur l et de valeur r

Exemple : Déplacer la classe

```
int x {100};          // x is an l-value

void func(int &num);  // A
void func(int &&num);   // B

func(x);          // calls A - x is an l-value
func(200);        // calls B - 200 is an r-value
```

Figure 42: OOP

```
class Move {
private:
   int *data;                      // raw pointer
public:
   void set_data_value(int d)      { *data = d; }
   int get data value()            { return *data; }
   Move(int d);                    // Constructor
   Move(const Move &source);       // Copy Constructor
   ~Move();                        // Destructor
};
```

Figure 43: OOP

Déplacer le constructeur de copie de classe

```
Move::Move(const Move &source) {
   data = new int;
   *data = *source.data;
}
```

Figure 44: OOP

- Allouer le stockage et la copie

Copie inefficace

- Les constructeurs de copie seront appelés pour copier les temps.

Qu'est ce que ça fait ? + Au lieu de faire une copie complète du constructeur de déplacement + déplace la ressource + copie simplement l'adresse de la ressource de la source vers l'objet actuel + Et, annule le pointeur dans le pointeur source

- Très efficace

syntaxe - référence de valeur r

```
Vector<Move> vec;

vec.push_back(Move{10});
vec.push_back(Move{20});
```

Figure 45: OOP

```
Constructor for: 10
Constructor for: 10
Copy constructor  - deep copy for: 10
Destructor freeing data for: 10
Constructor for: 20
Constructor for: 20
Copy constructor  - deep copy for: 20
Constructor for: 10
Copy constructor  - deep copy for: 10
Destructor freeing data for: 10
Destructor freeing data for: 20
```

Figure 46: OOP

```
Type::Type(Type &&source);

Player::Player(Player &&source);

Move::Move(Move &&source);
```

Figure 47: OOP

Constructeur de déplacement de classe de déplacement

```
class Move {
private:
   int *data;                  // raw pointer
public:
   void set_data_value(int d) { *data = d; }
   int get_data_value()           { return *data; }
   Move(int d);                // Constructor
   Move(const Move &source);   // Copy Constructor
   Move(Move &&source);        // Move Constructor
   ~Move();                // Destructor
};

Move::Move(Move &&source)
   : data{source.data} {
      source.data = nullptr;
}
```

- "voler" les données puis annuler le pointeur source

```
Vector<Move> vec;

vec.push_back(Move{10});
vec.push_back(Move{20});
```

efficace

- Les constructeurs de déplacement seront appelés pour les valeurs r temporaires.

```
Constructor for: 10
Move constructor - moving resource: 10
Destructor freeing data for nullptr
Constructor for: 20
Move constructor - moving resource: 20
Move constructor - moving resource: 10
Destructor freeing data for nullptr
Destructor freeing data for nullptr
Destructor freeing data for: 10
Destructor freeing data for: 20
```

Figure 48: OOP

Le pointeur `this`

pointeur `this`

- `this` est un mot-clé réservé
- Contient l'adresse de l'objet - c'est donc un pointeur vers l'objet.
- Ne peut être utilisé que dans le cadre d'une classe
- Tous les accès des membres se font via le pointeur `this`
- Peut être utilisé par le programmeur
  - Pour accéder aux données membres et aux méthodes
  - Pour le déterminer, deux objets sont identiques (plus d'informations dans la section suivante)
  - Peut être déférent `(*this)` pour donner l'objet actuel

```
void Account::set_balance(double bal) {
   balance = bal;   // this->balance is implied
}
```

Figure 49: OOP

- Pour lever l'ambiguïté, utilisez l'identifiant

```
void Account::set_balance(double balance) {
   balance = balance;  // which balance? The parameter
}

void Account::set_balance(double balance) {
   this->balance = balance;   // Unambiguous
}
```

Figure 50: OOP

Pour déterminer l'identité de l'objet

- Parfois, il est utile de savoir si deux objets sont le même objet

```
int Account::compare_balance(const Account &other) {
   if (this == &other)
      std::cout << "The same objects" << std::endl;
   ...
}
frank_account.compare_balance(frank_account);
```

Figure 51: OOP

- Nous utiliserons à nouveau le pointeur `this` lorsque nous surchargerons l'opérateur d'affectation

Utiliser const avec les classes

- Passer les arguments à la méthode membre de la classe en tant que `const`
- On peut aussi créer des objets `const`
- Que se passe-t-il si nous appelons une fonction membre sur des objets `const` ?
- `const`-exactitude

Création d'un objet const + `villain` est un objet const donc ses attributs ne

```
const Player villain {"Villain", 100, 55};
```

peuvent pas changer

- Que se passe-t-il lorsque nous appelons des méthodes membres sur un objet const ?

```
const Player villain {"Villain", 100, 55};

villain.set_name("Nice guy");                // ERROR

std::cout << villain.get_name() << std::endl; // ERROR
```

```
const Player villain {"Villain", 100, 55};

void display_player_name(const Player &p) {
   cout << p.get_name() << endl;
}

display_player_name(villain);          // ERROR
```

Utiliser const avec des classes + méthodes `const`

```
class Player {
  private:
     . . .
  public:
     std::string get_name() const;
     . . .
  };
```

Figure 52: OOP

- const-exactitude
- méthodes `const`

```
const Player villain {"Villain", 100, 55};

villain.set_name("Nice guy");                // ERROR

std::cout << villain.get_name() << std::endl; // OK
```

Figure 53: OOP

```
class Player {
  private:
     . . .
  public:
     std::string get_name() const;
     // ERROR if code in get_name modifies this object
     . . .
  };
```

Figure 54: OOP

Membres de la classe statique

- Les données membres de classe peuvent être déclarées comme statiques
  - Une seule donnée membre qui appartient à la classe, pas aux objets
  - Utile pour stocker des informations à l'échelle de la classe
- Les fonctions de classe peuvent être déclarées comme statiques
  - Indépendant de tout objet
  - Peut être appelé en utilisant le nom de la classe

Classe de joueur - membres `static`

```
class Player {
private:
   static int num_players;

public:
   static int get_num_players();
   . . .
};
```

Figure 55: OOP

Classe de joueur - initialise les données « statiques » + Généralement dans `Player.cpp`

```
#include "Player.h"

int Player::num_players = 0;
```

Figure 56: OOP

Classe Player - Méthode statique implémentée

```
int Player::get_num_players() {
    return num_players;
}
```

Figure 57: OOP

Classe Player - mettre à jour le constructeur

```
Player::Player(std::string name_val, int health_val,
int xp_val)
    : name{name_val}, health{health_val}, xp{xp_val} {
        ++num_players;
}
```

Figure 58: OOP

Classe de joueur - Destructeur

main.cpp

Structures vs Classes

- En plus de définir une `classe`, nous pouvons déclarer une `struct`.
- `struct` vient du langage de programmation `C`
- Essentiellement la même chose qu'une « classe »
  - Les membres sont « publics » par défaut

```
Player::~Player() {
    --num_players;
}
```

Figure 59: OOP

```
void display_active_players() {
   cout << "Active players: "
        << Player::get_num_players() << endl;
}

int main() {
   display_active_players();

   Player obj1 {"Frank"};
   display_active_players();
   . . .
```

Figure 60: OOP

```
class Person {
   std::string name;
   std::string get_name();
};

Person p;
p.name = "Frank";       // compiler error - private
std::cout << p.get_name(); // compiler error - private
```

classe

```
struct Person {
   std::string name;
   std::string get_name(); // Why if name is public?
};

Person p;
p.name = "Frank";       // OK - public
std::cout << p.get_name(); // OK - public
```

structure

Quelques directives générales + **structure** + Utilisez `struct` pour les objets passifs avec accès public + Ne déclarez pas de méthodes dans `struct`

- **classe**

  - Utiliser la classe pour les objets actifs avec accès privé
  - Implémentez les « getters/setters » si nécessaire
  - Implémenter les méthodes membres selon les besoins

Ami d'une classe

- **`Friend`**
  - Une fonction ou une classe qui a accès au membre de la classe privée
  - Et, cette fonction ou cette classe n'est « PAS » un membre de la classe à laquelle elle accède
- **`Fonction`**
  - Peut être des fonctions régulières de non-membres
  - Peut être des méthodes membres d'une autre classe
- **`Classe`**
  - Une autre classe peut avoir accès aux membres de la classe privée.
- L'amitié doit être accordée PAS prise
  - Déclaré explicitement dans la classe qui accorde l'amitié
  - Déclaré dans le prototype de fonction avec le mot-clé `friend`
- L'amitié n'est pas symétrique
  - Doit être explicitement accordé
    * Si A est un ami de B
    * B n'est PAS un ami de A
- L'amitié n'est pas transitive
  - Doit être accordé explicitement
    * Si A est un ami de B ET
    * B est un ami de C
    * alors A n'est PAS un ami de C

```
class Player {
   friend void display_player(Player &p);
   std::string name;
   int health;
   int xp;
public:
   . . .
};
```

fonction non membre

```
void display_player(Player &p) {
   std::cout << p.name << std::endl;
   std::cout << p.health << std::endl;
   std::cout << p.xp << std::endl;
}
```

- `display_player` peut également modifier les membres de données privées.

```
class Player {
   friend void Other_class::display_player(Player &p);
   std::string name;
   int health;
   int xp;
public:
   . . .
};
```

fonction membre d'une autre classe

```
class Other_class {
   . . .
public:
   void display_player(Player &p) {
      std::cout << p.name << std::endl;
      std::cout << p.health << std::endl;
      std::cout << p.xp << std::endl;
   }
};
```

```
class Player {
   friend class Other_class;
   std::string name;
   int health;
   int xp;
public:
   . . .
};
```

Un autre cours en tant qu'ami